

Manuale Desktop Bridge — 108 Vision AI

****Versione**:** 1.0 — 9 giugno 2026 ****Autore**:** Elios Scoglio ****Stato**:** Fase 3.5 completata, Fase 8 (macOS) pianificata

Indice

- Visione Generale
- Architettura Tecnica
- Modello di Sicurezza
- Capacità Desktop — Le 12 Azioni
- Casi d'Uso Reali
- Configurazione e Setup
- Dashboard del Consulente
- Client macOS — Roadmap Fase 8
- Vantaggi per il Cliente
- Limitazioni e Roadmap

1. Visione Generale

Cosa fa il Desktop Bridge

Il Desktop Bridge è il layer software che permette all'agente AI locale — installato sul computer del dipendente — di interagire con tutte le finestre aperte sul sistema operativo. Non è un plugin per una singola applicazione: è un ponte universale tra l'intelligenza artificiale e l'intero ambiente di lavoro digitale del cliente.

Concretamente, il Desktop Bridge fa tre cose:

- **Percepisce** — Legge il contenuto di qualsiasi finestra aperta: testo, struttura UI, bottoni, campi form, immagini. Capisce quale applicazione è in primo piano, cosa sta guardando l'utente, qual è il contesto operativo in quel momento.
- **Ragiona** — Incrocia ciò che vede con la knowledge base aziendale del cliente (documenti interni, procedure, CRM, prodotti). L'agente non risponde nel vuoto: risponde in contesto, sapendo che l'utente sta guardando una specifica email di un cliente specifico, o un preventivo per un prodotto specifico.
- **Agisce** — Se autorizzato (dal sistema di approvazione o per azioni a basso rischio già pre-approvate), può interagire attivamente con le finestre: scrivere testo, cliccare bottoni, compilare form, eseguire shortcut da tastiera.

Il risultato è un assistente AI che non vive in una tab del browser separata, ma è integrato nel flusso di lavoro reale del dipendente.

Prodotti simili esistono: Claude Desktop di Anthropic, ChatGPT Desktop di OpenAI. Entrambi offrono "computer use". La differenza di 108 Vision AI è il contesto aziendale, il controllo del consulente e il modello di governance — descritti in dettaglio nella sezione comparativa qui sotto.

Perché è rivoluzionario per le PMI

Le PMI hanno un problema specifico con l'adozione AI: i dipendenti non cambiano i loro strumenti di lavoro per usare l'AI. Aprire un chatbot separato, copincollare il testo di un'email, ricevere una risposta, ricopincollare — questo processo è cognitivamente costoso e viene abbandonato dopo pochi giorni.

Il Desktop Bridge elimina questo problema alla radice. L'AI viene dove sta il lavoro, non il contrario.

Prima del Desktop Bridge:

Dipendente riceve email da cliente
 → Apre chatbot AI
 → Copia manualmente il testo dell'email nel chatbot
 → Chiede "come rispondo?"
 → Riceve risposta generica (il chatbot non conosce il contesto cliente)
 → Adatta manualmente la risposta
 → Torna su Outlook
 → Incolla e modifica
 → Tempo totale: 4-6 minuti per una risposta

Con il Desktop Bridge:

Dipendente riceve email da cliente
 → L'agente legge l'email in background (o su richiesta)
 → Incrocia con KB: storico cliente, prodotti acquistati, procedure aziendali
 → Suggerisce bozza di risposta nella barra laterale dell'agente
 → Se approvata, la scrive direttamente nel campo di risposta Outlook
 → Tempo totale: 40 secondi

Questo non è solo risparmio di tempo. È eliminazione del context switching — il cambio continuo tra applicazioni che studi di produttività identificano come uno dei principali consumatori di energia cognitiva nei lavoratori della conoscenza.

I benefici concreti per una PMI tipica:

- Riduzione del 60-70% del tempo su email di risposta standard
- Eliminazione quasi totale degli errori di data entry (l'agente copia, non il dipendente)
- Risposta più rapida e consistente ai clienti (la KB garantisce uniformità)
- I nuovi dipendenti diventano operativi molto prima (l'agente compensa la mancanza di conoscenza del contesto)

Differenziatori competitivi

Caratteristica | 108 Vision AI | Claude Desktop | ChatGPT Desktop | Microsoft Copilot

Lettura finestre desktop | SI | SI | SI | Parziale (solo app MS)
 Azioni su finestre | SI | SI | SI | Parziale
 Knowledge base aziendale | SI (core feature) | NO | NO | Parziale (SharePoint)
 Multi-tenant | SI | NO | NO | NO (single tenant)
 Approvazione remota consulente | SI | NO | NO | NO
 Funzionamento always-on background | SI | Parziale | Parziale | SI (solo MS 365)
 Audit trail con screenshot | SI | NO | NO | Parziale
 Controllo per processo/app | SI | Limitato | Limitato | NO
 Rate limiting configurabile | SI | NO | NO | NO
 Deployment on-premise / data residency | Roadmap | NO | NO | SI (ma costo enterprise)
 Prezzo per PMI | Accessibile | Costoso (Claude Pro) | Costoso | Costoso (M365 E3/E5)

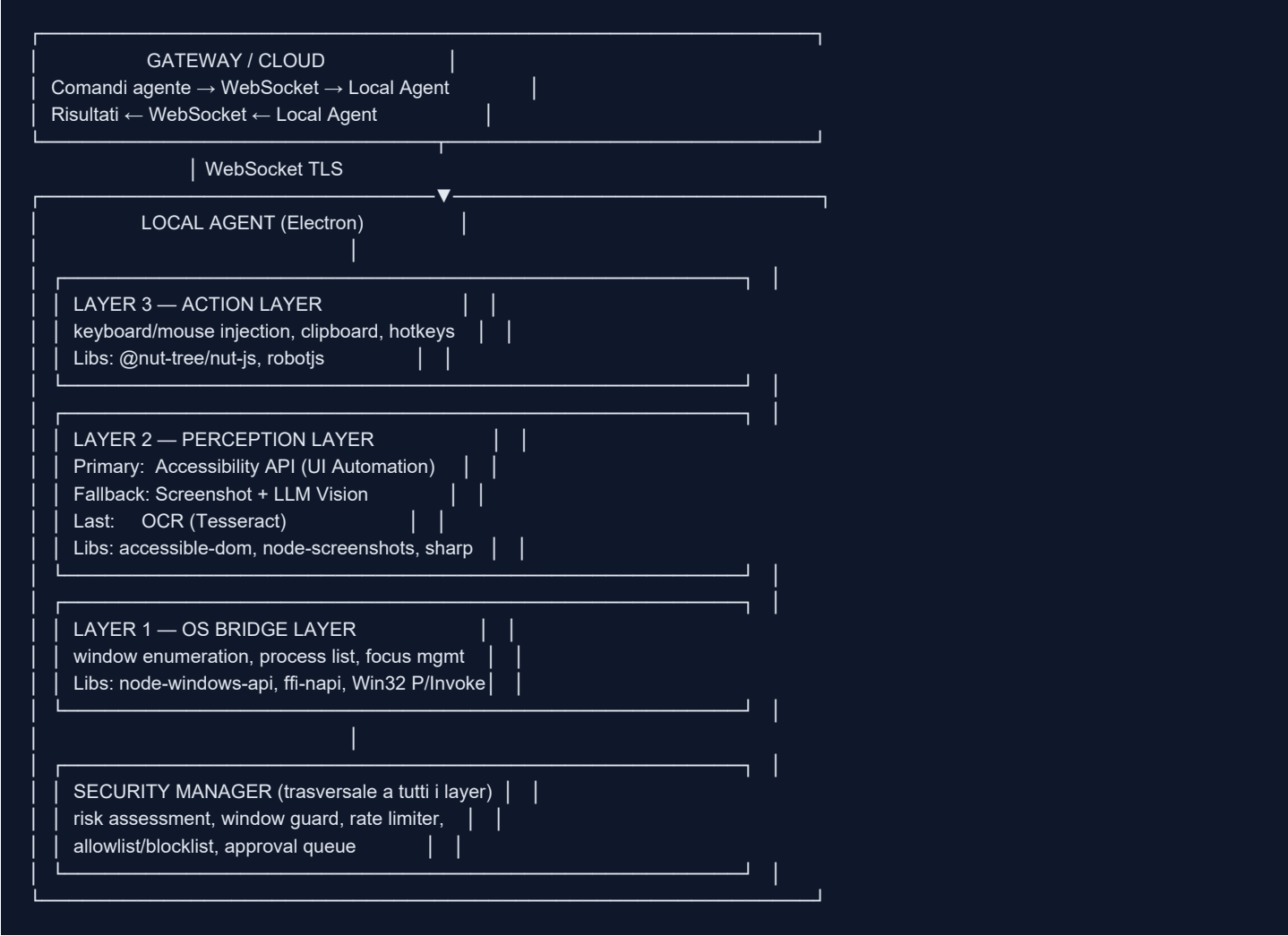
Il differenziatore più importante non è tecnologico: è il modello di governance. Claude Desktop e ChatGPT Desktop danno controllo diretto all'utente finale sull'agente. In un contesto PMI, questo è problematico: un dipendente potrebbe autorizzare azioni errate, l'agente potrebbe fare danni senza che il responsabile lo sappia.

Il modello 108 Vision AI inverte la logica: il consulente esterno (o il responsabile interno) mantiene supervisione sulle azioni ad alto rischio, con un sistema di approvazione che funziona in tempo reale. L'utente finale beneficia dell'automazione, ma non porta il peso della governance.

2. Architettura Tecnica

Schema a livelli

Il Desktop Bridge è organizzato in tre strati funzionali sovrapposti, ciascuno con responsabilità distinte:



Layer 1 — OS Bridge: Interfaccia con le API native del sistema operativo. Su Windows usa Win32 API tramite ffi-napi (foreign function interface per Node.js) per enumerare le finestre aperte (EnumWindows), recuperare titoli (GetWindowText), handle di processo (GetWindowThreadProcessId), coordinate e dimensioni (GetWindowRect). Questo layer non legge contenuti: sa solo quali finestre esistono, dove sono, e a quale processo appartengono.

Layer 2 — Perception: Questo è il cuore intellettuale. Trasforma le finestre in informazione strutturata comprensibile dall'LLM. Usa tre strategie in cascata:

- Accessibility API (primario): Windows UI Automation espone un albero di elementi (AutomationElement) con proprietà come nome, tipo di controllo, valore, stato abilitato/disabilitato. È veloce, preciso, non richiede GPU, funziona anche con finestre in background.
- Vision LLM (fallback): Quando l'app non espone un buon accessibility tree (es. applicazioni legacy con interfaccia custom, app Electron mal configurate), si cattura uno screenshot e lo si invia all'LLM con vision capabilities. Più

lento e costoso, ma universale.

- OCR (ultimo resort): Per testi in immagini, PDF visualizzati in viewer custom, scan di documenti. Usa Tesseract.js locale, nessun dato inviato al cloud.

Layer 3 — Action: Trasforma i comandi ad alto livello in input fisici sul sistema operativo. Simula pressioni di tasto, movimenti e click del mouse, operazioni clipboard. Usa `@nut-tree/nut-js` come libreria principale per la compatibilità cross-platform (utile per la futura transizione macOS).

Security Manager: Non è un layer separato ma un componente trasversale che viene consultato prima di qualsiasi operazione. Valuta il rischio di ogni azione, gestisce la coda di approvazione, verifica l'identità della finestra target, applica rate limiting.

Approccio Hybrid: Accessibility + Vision + OCR

La scelta dell'approccio ibrido non è arbitraria. Ogni metodo ha vantaggi e limiti precisi:

Accessibility API — vantaggi:

- Struttura dati semantica (sa che un elemento è un "campo email", non solo un rettangolo con testo)
- Funziona senza screenshot (no GPU, no latenza render)
- Lettura precisa anche di testo che non è visibile sullo schermo (es. elementi scrollati fuori vista)
- Interazione precisa senza coordinate assolute (clicca su "quel bottone", non su "coordinate 450,320")

Accessibility API — limiti:

- Applicazioni legacy in Win32 puro possono avere alberi poveri o assenti
- App Java/Swing spesso non espongono attributi corretti
- App custom con rendering diretto su canvas sono cieche per l'accessibilità

Vision LLM — vantaggi:

- Universale: funziona su qualsiasi app
- Comprende contesto visivo complesso (tabelle, layout, colori)
- Può identificare elementi senza nome nel UI tree

Vision LLM — limiti:

- Costo token per ogni screenshot
- Latenza: 1-3 secondi per interpretazione
- Richiede invio screenshot al cloud (implicazioni privacy)
- Coordinate stimate, non esatte (rischio click errato)

OCR — vantaggi:

- Locale, nessun dato al cloud
- Ottimo per testi in immagini, PDF rasterizzati

OCR — limiti:

- Solo testo, nessuna struttura semantica
- Accuratezza variabile con font non standard, bassa risoluzione

La strategia di fallback è automatica: il Local Agent prova Accessibility, se la qualità del tree è sotto una soglia configurabile (es. meno di N elementi con nome, o nessun elemento interattivo trovato), passa a Vision. Se il task richiede solo lettura di testo e Vision non è disponibile o disabilitata per privacy, usa OCR.

Flusso di un'azione tipica

Di seguito il flusso completo per un'azione high-risk (es. compilare un campo in un form CRM):

```
Step 1: COMANDO IN INGRESSO
Gateway Cloud → WebSocket TLS → Local Agent
Payload: {
  action: "desktop.typeText",
  params: { windowTitle: "Preventivo #2341 — CRM", field: "Nome cliente", text: "Mario Rossi" },
  riskLevel: "high",
  requestedBy: "agent-session-abc123"
}

Step 2: SECURITY PRE-CHECK
Security Manager valuta:
- riskLevel è "high" → richiede approvazione consulente
- Processo nella allowlist? → verifica config.json
- Rate limit non superato? → verifica contatore azioni/minuto
Se uno dei check fallisce → risposta immediata "BLOCKED" al gateway

Step 3: IDENTIFICAZIONE FINESTRA
OS Bridge Layer cerca finestra con titolo corrispondente
→ Trova HWND 0x003A1C (handle Win32)
→ Verifica processo: "crm_app.exe" → in allowlist
→ Acquisisce coordinate: {x:100, y:50, w:1200, h:800}

Step 4: SCREENSHOT PRE-AZIONE
Perception Layer cattura screenshot della finestra target
→ Salvato come pre_action_20260609_143201.png
→ Inviato al gateway per audit trail

Step 5: NOTIFICA CONSULENTE
Gateway invia push notification alla dashboard del consulente:
"Azione richiesta: Scrivi 'Mario Rossi' nel campo 'Nome cliente' del preventivo #2341"
+ Screenshot allegato
→ Consulente ha 60 secondi per approvare/rifiutare

Step 6: APPROVAZIONE RICEVUTA
Consulente clicca "Approva" nella dashboard
→ Gateway invia "APPROVED" al Local Agent

Step 7: WINDOW GUARD
Immediatamente prima dell'esecuzione, verifica:
- HWND ancora valido?
- Titolo finestra ancora corrispondente?
- Processo ancora lo stesso?
Se qualcosa è cambiato → ABORT (la finestra potrebbe essere cambiata nel frattempo)

Step 8: ESECUZIONE
Action Layer:
→ Porta la finestra in primo piano (FocusWindow)
→ Usa Accessibility API per trovare il campo "Nome cliente"
→ Click sul campo per dare focus
→ Type: "Mario Rossi" (con delay realistico tra i tasti)

Step 9: SCREENSHOT POST-AZIONE
Perception Layer cattura screenshot dopo l'azione
→ Salvato come post_action_20260609_143204.png
→ Comparato con pre-action per verifica visiva
```

Stack tecnologico

Componente	Libreria / Tecnologia	Versione	Scopo	Piattaforma
------------	-----------------------	----------	-------	-------------

Runtime container | Electron | 30.x | Shell cross-platform, system tray, IPC | Win/Mac/Linux
 OS Bridge Windows | ffi-napi + Win32 API | 4.x | EnumWindows, GetWindowText, HWND management | Windows
 OS Bridge macOS | osascript + AX API | — | AppleScript, Accessibility framework | macOS (Fase 8)
 Accessibility Windows | accessible-dom / UI Automation | — | Lettura UI tree strutturato | Windows
 Screenshot | node-screenshots | 0.2.x | Cattura finestra per handle | Win/Mac
 Image processing | sharp | 0.33.x | Resize, crop, conversione screenshot | Cross-platform
 OCR | tesseract.js | 5.x | Riconoscimento testo in immagini | Cross-platform
 Input simulation | @nut-tree/nut-js | 4.x | Keyboard/mouse injection | Win/Mac/Linux
 Input fallback | robotjs | 0.6.x | Fallback per azioni mouse | Win/Mac
 WebSocket client | ws | 8.x | Connessione permanente al gateway cloud | Cross-platform
 Security | Custom SecurityManager | interno | Risk assessment, approval queue, rate limit | Cross-platform
 Config | electron-store | 9.x | Persistenza config locale cifrata | Cross-platform
 Logging locale | winston | 3.x | Audit log locale, rotazione file | Cross-platform
 Auto-update | electron-updater | 6.x | Aggiornamento silenzioso agente | Cross-platform

3. Modello di Sicurezza

Tre livelli di rischio

Il modello di sicurezza del Desktop Bridge è costruito attorno a un principio semplice: non tutte le azioni hanno lo stesso impatto. Leggere una finestra è fondamentalmente diverso dal cliccare un bottone "Invia" su un'email. Il sistema categorizza ogni azione in tre livelli di rischio:

LIVELLO READ-ONLY

Azioni che non modificano alcuno stato. Leggono dati, fanno screenshot, analizzano strutture. Non richiedono approvazione. Vengono eseguite immediatamente.

Esempi:

- Leggere il testo di un'email aperta in Outlook
- Fare lo screenshot della finestra attiva
- Ottenere la lista delle finestre aperte
- Analizzare la struttura di un form (quali campi esistono, quali sono già compilati)
- Leggere il testo selezionato dall'utente

Rischio: praticamente zero. Il peggio che può succedere è che l'agente "veda" dati riservati. Questo viene mitigato a livello di blocklist (es. non leggere mai le finestre del gestore password).

LIVELLO LOW-RISK

Azioni che cambiano lo stato visivo ma non modificano dati persistenti. Portare una finestra in primo piano, scrollare, cambiare tab. Non richiedono approvazione se la configurazione del cliente prevede autonomia "standard".

Esempi:

- Portare in primo piano la finestra CRM
- Scrollare verso il basso in un documento
- Cambiare il focus tra campi di un form

- Copiare testo negli appunti (senza incollare)
- Aprire un link (navigazione solo lettura)

Rischio: basso. Un focus non autorizzato sulla finestra sbagliata disturba l'utente ma non causa danni. Il consulente può configurare questi livelli come "auto-approve" o "always ask" per ogni cliente.

LIVELLO HIGH-RISK

Azioni che modificano dati, inviano comunicazioni, eseguono operazioni irreversibili o potenzialmente irreversibili. Richiedono sempre approvazione esplicita del consulente (o del supervisor interno configurato), salvo che il consulente abbia pre-approvato pattern specifici.

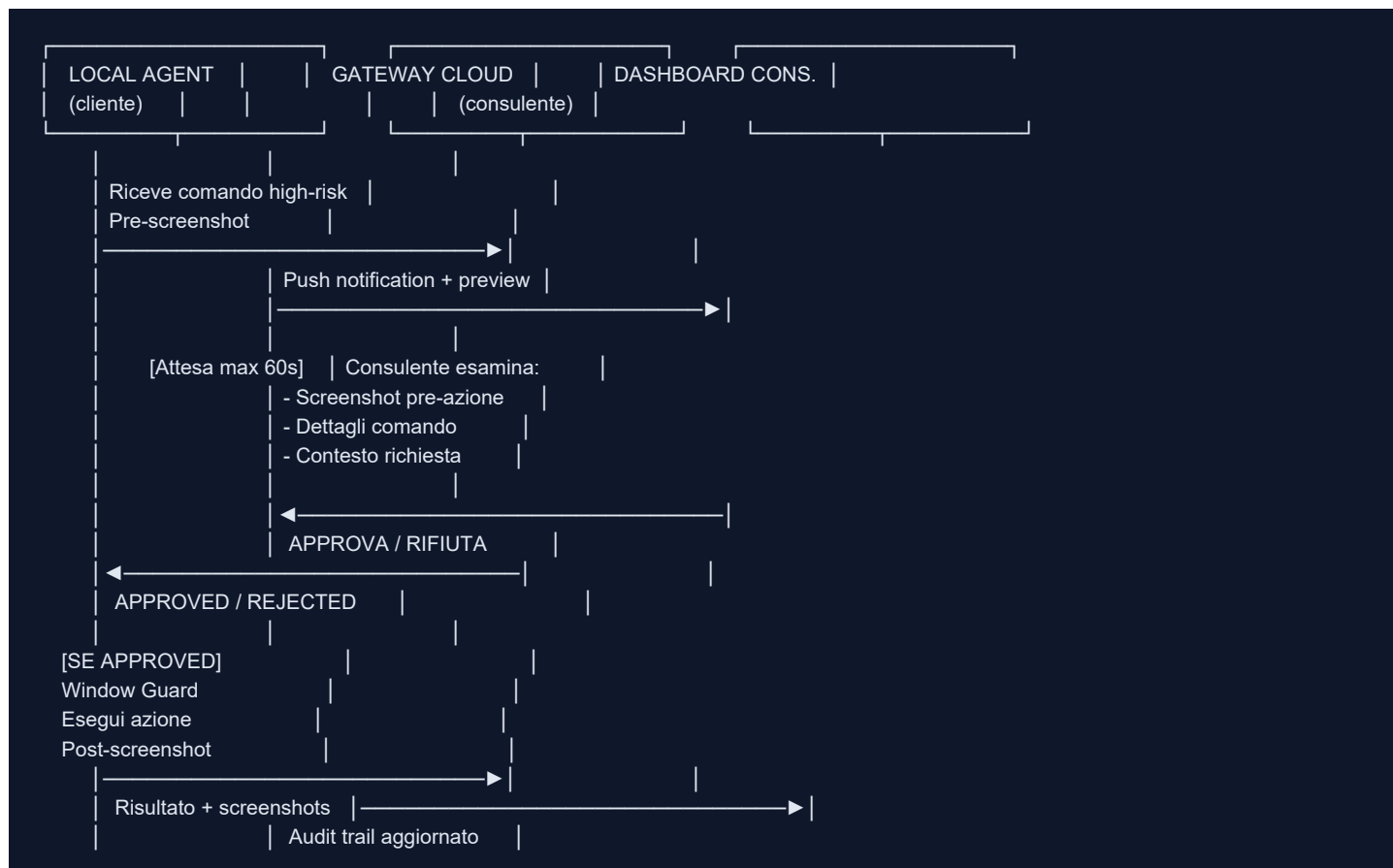
Esempi:

- Scrivere testo in un campo form (typeText)
- Cliccare un bottone (specialmente "Invia", "Salva", "Conferma", "Elimina")
- Eseguire una shortcut da tastiera che modifica dati (Ctrl+S, Ctrl+Z, Ctrl+Enter)
- Cliccare su link che aprono transazioni
- Qualsiasi azione su finestre di applicazioni finanziarie o gestionali

Rischio: alto. Un click sbagliato su "Invia" spedisce un'email al cliente con dati errati. Un Ctrl+S salva una versione corrotta di un documento. La firma digitale viene apposta. Questi scenari richiedono occhi umani prima dell'esecuzione.

Flusso di approvazione

Il sistema di approvazione è il meccanismo centrale che distingue 108 Vision AI da un semplice script di automazione. Funziona come segue:



Timeout e comportamento in caso di mancata risposta:

- Se il consulente non risponde entro 60 secondi (configurabile), l'azione viene automaticamente rifiutata e l'utente viene notificato che l'azione richiede approvazione manuale.
- Il consulente può configurare una "auto-approve list" di pattern ricorrenti. Esempio: "typeText in campi del CRM per clienti già esistenti" può essere pre-approvato, eliminando il ritardo per operazioni abituali.
- Il consulente può anche delegare l'approvazione a un supervisor interno del cliente per certi tipi di azioni.

Notifiche al consulente:

- Push notification su mobile (app 108 Vision AI)
- Notifica email per azioni non gestite entro 30 secondi
- Badge nella dashboard web

Window Guard

Il Window Guard risolve un problema sottile ma critico: il lag temporale tra quando un'azione viene approvata e quando viene eseguita. In quei 5-60 secondi di attesa approvazione, l'utente potrebbe aver cambiato applicazione, chiuso la finestra, o aperto una finestra diversa con lo stesso titolo.

Senza Window Guard: il consulente approva "clicca Invia nel preventivo #2341" ma nel frattempo l'utente ha minimizzato quella finestra e portato in primo piano l'email privata di un collega. L'agente clicca "Invia" sull'email privata.

Con Window Guard: immediatamente prima dell'esecuzione, il sistema verifica:


```

interface WindowGuardCheck {
  hwnd: number; // handle Win32 — univoco per processo di finestra
  title: string; // titolo finestra al momento dell'approvazione
  processName: string; // nome dell'eseguibile (es. "outlook.exe")
  processId: number; // PID
}

async function windowGuard(original: WindowGuardCheck): Promise<boolean> {
  const current = await getWindowInfo(original.hwnd);

  // Handle ancora valido?
  if (!current.exists) return false;

  // Stessa finestra per titolo?
  if (current.title !== original.title) return false;

  // Stesso processo?
  if (current.processId !== original.processId) return false;

  // Finestra ancora visibile e non minimizzata?
  if (current.isMinimized || current.isHidden) return false;

  return true;
}

```

Se il Window Guard fallisce, l'azione viene abortita con log di audit che spiega il motivo (WINDOW_CHANGED_AFTER_APPROVAL). Il consulente riceve notifica.

Screenshot Audit Trail

Per ogni azione high-risk eseguita o tentata, il sistema mantiene un audit trail completo con screenshot:

```

audit/
2026-06-09/
14-32-01_typeText_preventivo2341/
  pre_action.png    # Screenshot prima dell'azione
  post_action.png   # Screenshot dopo l'azione
  metadata.json     # Dettagli completi
  diff.png          # Differenza visiva (opzionale, compute-intensive)

```

Il file metadata.json contiene:

```
{
  "actionId": "act_20260609_143201_xK9mP",
  "sessionId": "sess_abc123",
  "agentVersion": "1.4.2",
  "timestamp": "2026-06-09T14:32:01Z",
  "action": "desktop.typeText",
  "params": {
    "windowTitle": "Preventivo #2341 — CRM",
    "field": "Nome cliente",
    "textLength": 11
  },
  "riskLevel": "high",
  "approvedBy": "mario.consulente@108vision.it",
  "approvalTimestamp": "2026-06-09T14:32:03Z",
  "approvalLatencyMs": 1847,
  "windowGuardResult": "PASSED",
  "executionResult": "SUCCESS",
  "executionTimeMs": 1240,
  "preScreenshot": "pre_action.png",
  "postScreenshot": "post_action.png",
  "clientId": "cliente_xyz",
  "tenantId": "tenant_108vision"
}
```

Gli screenshot vengono:

- Conservati localmente per 30 giorni (configurabile)
- Caricati cifrati nel cloud per audit esteso (90 giorni, configurabile)
- Accessibili solo al consulente assegnato e all'admin della piattaforma
- Mai visibili all'agente AI stesso nelle chiamate successive (per evitare che informazioni sensibili degli screenshot entrino in prompt futuri)

Allowlist / Blocklist processi

Il consulente può configurare, per ogni cliente, quali processi/applicazioni l'agente può toccare:

```
// config.json — sezione desktopAccess
{
  "desktopAccess": {
    "enabled": true,
    "processAllowList": [
      "outlook.exe",
      "chrome.exe",
      "crm_pro.exe",
      "word.exe",
      "excel.exe"
    ],
    "processBlockList": [
      "keepass.exe",
      "1password.exe",
      "bitwarden.exe",
      "taskmgr.exe",
      "regedit.exe",
      "powershell.exe",
      "cmd.exe"
    ],
    "strictMode": false
  }
}
```

Comportamento:

- Se `strictMode: false` → solo i processi in `processBlockList` sono vietati; tutto il resto è consentito (utile durante onboarding quando non si conoscono ancora tutte le app usate).
- Se `strictMode: true` → solo i processi in `processAllowList` sono consentiti; tutto il resto è bloccato (modalità raccomandata per ambienti con dati sensibili).
- Le applicazioni nella blocklist ricevono sempre risposta `BLOCKED` senza neanche tentare la lettura.

Nota: il blocklist include sempre e per default i gestori di password (keepass.exe, 1password.exe, bitwarden.exe, dashlane.exe) e i tool di sistema (taskmgr.exe, regedit.exe, cmd.exe, powershell.exe). Questa blocklist di sistema non può essere rimossa dal consulente, solo il team 108 Vision AI può modificarla in casi eccezionali.

Rate Limiting

Per prevenire loop impazziti, errori di configurazione che generano migliaia di azioni in sequenza, o potenziali attacchi, il sistema applica rate limiting a più livelli:

Livello	Limite Default	Finestra	Comportamento al superamento
Azioni totali	30/min	1 minuto	Sospensione temporanea + alert consulente
Azioni high-risk	10/min	1 minuto	Blocco + alert immediato
Azioni sullo stesso processo	20/min	1 minuto	Throttling (rallentamento)
Screenshot consecutivi	5 in 10s	—	Pause forzata 30s
Approvazioni pendenti	3 simultanee	—	Nuove richieste messe in coda

Tutti i limiti sono configurabili dal consulente nella dashboard. Per clienti con workflow ad alto volume (es. data entry intensivo), i limiti possono essere aumentati previa valutazione.

4. Capacità Desktop — Le 12 Azioni

Azioni Read-Only

`desktop.listWindows`

Descrizione: Restituisce la lista di tutte le finestre visibili aperte sul desktop.

Parametri: Nessuno (opzionalmente includeMinimized: boolean)

Risk Level: read-only

Cosa ritorna:

```
{
  "windows": [
    {
      "hwnd": "0x003A1C",
      "title": "Preventivo #2341 — CRM Pro",
      "processName": "crm_pro.exe",
      "processId": 12847,
      "isMinimized": false,
      "rect": { "x": 100, "y": 50, "width": 1200, "height": 800 },
      "isFocused": true
    },
    {
      "hwnd": "0x001B4E",
      "title": "Posta in arrivo — Outlook",
      "processName": "outlook.exe",
      "processId": 9234,
      "isMinimized": false,
      "rect": { "x": 0, "y": 0, "width": 1920, "height": 1080 },
      "isFocused": false
    }
  ],
  "count": 2,
  "timestamp": "2026-06-09T14:30:00Z"
}
```

Esempio d'uso: L'agente vuole sapere se il CRM è aperto prima di suggerire di inserire dati nel CRM.

`desktop.readWindow`

Descrizione: Legge il contenuto testuale e la struttura UI di una finestra specifica, identificata da handle, titolo o processo.

Parametri:

- target: { hwnd?: string, title?: string, processName?: string } — almeno uno dei tre
- method: "accessibility" | "vision" | "ocr" | "auto" (default: "auto")
- includeHidden: boolean (default: false) — include elementi non visibili sullo schermo

Risk Level: read-only

Cosa ritorna:

```
{
  "windowTitle": "Preventivo #2341 — CRM Pro",
  "method": "accessibility",
  "text": "Preventivo #2341\nCliente: [vuoto]\nProdotto: Licenza Enterprise\nPrezzo: €2.400,00\n...",
  "elements": [
    {
      "name": "Nome cliente",
      "type": "Edit",
      "value": "",
      "isEnabled": true,
      "rect": { "x": 340, "y": 220, "width": 280, "height": 32 }
    },
    {
      "name": "Salva",
      "type": "Button",
      "value": null,
      "isEnabled": true,
      "rect": { "x": 900, "y": 720, "width": 120, "height": 40 }
    }
  ],
  "screenshotUrl": null
}
```

Esempio d'uso: L'agente legge il preventivo aperto per capire quali campi mancano e suggerire i valori dalla KB cliente.

`desktop.readFocused`

Descrizione: Shortcut per leggere la finestra attualmente in primo piano. Equivale a `desktop.readWindow` con target automaticamente risolto alla finestra focalizzata.

Parametri:

- `method`: "accessibility" | "vision" | "ocr" | "auto" (default: "auto")

Risk Level: read-only

Cosa ritorna: Stesso schema di `desktop.readWindow`.

Esempio d'uso: L'utente chiede all'agente "cosa vedi?" o "aiutami con questo" — l'agente legge immediatamente ciò che l'utente sta guardando.

`desktop.screenshot`

Descrizione: Cattura uno screenshot della finestra specificata o dell'intero schermo.

Parametri:

- `target`: { `hwnd?`: string, `title?`: string, `processName?`: string, `fullScreen?`: boolean }
- `quality`: "low" | "medium" | "high" (default: "medium", influisce su dimensione file)
- `redact`: string[] — lista di pattern di testo da oscurare (per privacy)

Risk Level: read-only

Cosa ritorna:

```
{
  "screenshotId": "scr_20260609_143000_ABC",
  "url": "https://storage.108vision.ai/tenants/xyz/screenshots/scr_20260609_143000_ABC.png",
  "width": 1200,
  "height": 800,
  "timestamp": "2026-06-09T14:30:00Z",
  "fileSizeBytes": 284720
}
```

Esempio d'uso: Il consulente vuole vedere cosa sta guardando il cliente in un momento specifico durante una sessione di supporto remoto.

`desktop.analyzeScreen`

Descrizione: Combina screenshot e analisi LLM per ottenere una descrizione semantica di ciò che è visibile. Più costoso di readWindow ma più universale.

Parametri:

- target: stesso schema di desktop.screenshot
- question: string — domanda specifica da fare sul contenuto visivo (es. "quali campi sono ancora vuoti?")
- model: "fast" | "accurate" (default: "fast")

Risk Level: read-only

Cosa ritorna:

```
{
  "analysis": "La finestra mostra un form di preventivo. I campi 'Nome cliente' e 'Email' sono vuoti. Il campo 'Prodotto'",
  "confidence": 0.91,
  "screenshotId": "scr_20260609_143015_DEF",
  "tokensUsed": 1240,
  "modelUsed": "claude-haiku-4"
}
```

Esempio d'uso: App legacy senza accessibility tree decente. L'agente deve capire cosa c'è sullo schermo prima di agire.

`desktop.getUITree`

Descrizione: Restituisce l'albero completo degli elementi UI di una finestra secondo l'Accessibility API. Utile per debug e per la costruzione di automazioni precise.

Parametri:

- target: { hwnd?: string, title?: string, processName?: string }
- depth: number (default: 5) — profondità massima dell'albero da esplorare
- filter: "all" | "interactive" (default: "interactive") — filtra solo gli elementi interattivi

Risk Level: read-only

Cosa ritorna: Struttura ad albero JSON degli elementi AutomationElement.

Esempio d'uso: Il consulente o l'agente vuole esplorare la struttura di un'applicazione per pianificare automazioni future o debuggare un'azione che non trova il campo corretto.

Azioni Low-Risk

`desktop.focusWindow`

Descrizione: Porta una finestra in primo piano, rendendola la finestra attiva e visibile all'utente.

Parametri:

- target: { hwnd?: string, title?: string, processName?: string }
- restore: boolean (default: true) — se minimizzata, la ripristina

Risk Level: low

Cosa ritorna:

```
{ "success": true, "windowTitle": "CRM Pro — Preventivo #2341", "wasMinimized": false }
```

Esempio d'uso: L'agente ha analizzato un'email e vuole portare in primo piano il CRM per guidare l'utente nell'inserimento dei dati. Non agisce da solo: porta la finestra in vista e poi aspetta conferma dell'utente o approvazione per azioni successive.

`desktop.scrollWindow`

Descrizione: Esegue uno scroll verticale o orizzontale su una finestra o su un elemento specifico al suo interno.

Parametri:

- target: { hwnd?: string, title?: string, processName?: string }
- direction: "up" | "down" | "left" | "right"
- amount: "page" | "half" | number (numero di righe, default: 3)
- element: string (opzionale — nome dell'elemento scrollabile se diverso dalla finestra)

Risk Level: low

Cosa ritorna:

```
{ "success": true, "scrolled": true, "direction": "down", "amount": 3 }
```

Esempio d'uso: L'agente sta leggendo un lungo documento Word per trovare una sezione specifica. Scrolla automaticamente senza disturbare il lavoro dell'utente.

Azioni High-Risk

`desktop.typeText`

Descrizione: Scrive testo in un campo di input specifico di una finestra. Il campo può essere identificato per nome (accessibilità), placeholder, o posizione.

Parametri:

- target: { hwnd?: string, title?: string, processName?: string }
- field: string — nome o identificatore del campo di input (da UI tree o visivo)
- text: string — testo da scrivere
- clearFirst: boolean (default: false) — cancella il contenuto esistente prima di scrivere
- typingSpeed: "instant" | "realistic" | "slow" (default: "realistic") — instant = Clipboard paste, realistic = simulazione digitazione umana

Risk Level: high

Cosa ritorna:

```
{
  "success": true,
  "field": "Nome cliente",
  "textWritten": "Mario Rossi",
  "method": "accessibility",
  "preScreenshotId": "scr_pre_143201",
  "postScreenshotId": "scr_post_143204"
}
```

Esempio d'uso: Compilare automaticamente un form CRM con dati estratti da un documento aperto.

`desktop.clickElement`

Descrizione: Clicca su un elemento UI specifico (bottone, link, checkbox, voce di menu) identificato per nome o tipo.

Parametri:

- target: { hwnd?: string, title?: string, processName?: string }
- element: string — nome del bottone/elemento da cliccare
- elementType: "button" | "link" | "checkbox" | "menuItem" | "any" (default: "any")
- confirm: boolean (default: false) — richiede conferma ulteriore dell'utente locale prima del click

Risk Level: high

Cosa ritorna:

```
{
  "success": true,
  "elementClicked": "Salva preventivo",
  "elementType": "button",
  "coordinates": { "x": 960, "y": 740 },
  "preScreenshotId": "scr_pre_143501",
  "postScreenshotId": "scr_post_143503"
}
```

Nota critica: Il parametro confirm aggiunge un secondo livello di sicurezza — oltre all'approvazione del consulente, l'utente locale deve confermare (tramite dialog nella tray icon) prima che il click venga eseguito. Raccomandato per click su bottoni come "Invia email", "Conferma ordine", "Elimina".

Esempio d'uso: Dopo aver compilato un form preventivo, cliccare "Salva" per persistere i dati.

`desktop.pressHotkey`

Descrizione: Esegue una combinazione di tasti (shortcut da tastiera) nella finestra specificata.

Parametri:

- target: { hwnd?: string, title?: string, processName?: string }
- keys: string[] — lista di tasti da premere in combinazione (es. ["ctrl", "s"], ["alt", "F4"])
- sequential: boolean (default: false) — se true, i tasti vengono premuti in sequenza invece che simultaneamente

Risk Level: high

Cosa ritorna:

```
{
  "success": true,
  "keysCombination": ["ctrl", "s"],
  "preScreenshotId": "scr_pre_144001",
  "postScreenshotId": "scr_post_144003"
}
```

Esempio d'uso: Salvare un documento Word con Ctrl+S dopo modifiche automatiche, oppure navigare tra tab con Ctrl+Tab.

`desktop.mouseClick`

Descrizione: Esegue un click del mouse a coordinate assolute o relative a una finestra. Meno preciso di clickElement ma necessario per app senza accessibility tree.

Parametri:

- target: { hwnd?: string, title?: string, processName?: string }
- x: number — coordinata X (relativa alla finestra se relative: true)
- y: number — coordinata Y
- relative: boolean (default: true) — coordinate relative alla finestra vs schermo
- button: "left" | "right" | "double" (default: "left")

Risk Level: high

Cosa ritorna:

```
{
  "success": true,
  "coordinates": { "x": 340, "y": 220, "relative": true },
  "button": "left",
  "preScreenshotId": "scr_pre_144201",
  "postScreenshotId": "scr_post_144203"
}
```

Avviso: Questa azione è la più pericolosa perché le coordinate cambiano se la finestra viene ridimensionata o spostata. Usare clickElement quando possibile. mouseClick dovrebbe essere l'ultimo resort per app incompatibili con Accessibility API.

5. Casi d'Uso Reali

Caso 1: Assistente Email Intelligente

Scenario: Un'azienda B2B riceve ogni giorno 40-60 email da clienti con richieste di informazioni su prezzi, disponibilità prodotti, stato ordini. Il team commerciale perde 2-3 ore al giorno a rispondere.

Come funziona con il Desktop Bridge:

- Il dipendente apre un'email in Outlook (o Thunderbird). L'agente, in esecuzione in background, rileva il cambio di finestra focalizzata.
- Su richiesta esplicita del dipendente ("aiutami a rispondere a questa") oppure automaticamente se configurato per le email dai clienti noti, l'agente esegue `desktop.readFocused` per leggere il contenuto dell'email.
- L'agente incrocia il contenuto con la KB aziendale: cerca il cliente per nome/email, recupera storico ordini, cerca nei listini prodotti menzionati, verifica disponibilità.
- Genera una bozza di risposta contestualizzata, visualizzata nella barra laterale dell'agente locale.
- Il dipendente legge la bozza, la approva con un click o la modifica.
- Se la bozza è approvata, l'agente esegue `desktop.focusWindow` su Outlook, poi `desktop.clickElement` su "Rispondi", poi `desktop.typeText` nel campo di risposta con la bozza.
- L'azione `typeText` è high-risk e richiede approvazione del consulente — o, se il consulente ha pre-approvato il pattern "typeText in risposta email Outlook", viene eseguita direttamente.
- Il dipendente rivede il testo già inserito nel campo di risposta e preme Invia manualmente (il click su Invia non viene automatizzato senza doppia conferma).

Risultato: Da 5-7 minuti per risposta a 60-90 secondi. Qualità e consistenza della risposta migliorano perché l'agente conosce lo storico del cliente e i prodotti meglio di quanto li ricordi il dipendente.

Caso 2: Compilazione Automatica CRM

Scenario: Un consulente riceve un preventivo via email in formato PDF allegato. Deve trasferire manualmente i dati (ragione sociale, partita IVA, prodotti, quantità, prezzi) nel CRM web. Operazione che richiede 10-15 minuti e introduce frequentemente errori di trascrizione.

Come funziona con il Desktop Bridge:

- Il dipendente apre il PDF allegato nel viewer (Adobe Acrobat, Edge PDF viewer, ecc.). Dice all'agente: "estrai i dati da questo preventivo e mettili nel CRM".
- L'agente esegue `desktop.readWindow` con fallback a `vision` o `ocr` sul viewer PDF. Estrae: ragione sociale, PIVA, indirizzo, prodotti con quantità e prezzi.
- L'agente verifica in KB se il cliente esiste già nel CRM o è nuovo. Se è nuovo, prepara un set di dati per la creazione.
- L'agente esegue `desktop.focusWindow` sul browser con il CRM aperto, naviga alla sezione "Nuovo cliente" o "Nuovo preventivo".
- Per ogni campo: `desktop.clickElement` sul campo, poi `desktop.typeText` con il valore estratto. Ogni coppia `click+type` è un'azione high-risk.
- Il consulente approva le azioni dalla dashboard — o, se ha configurato "auto-approve per data entry CRM da PDF", il flusso è automatico con audit trail completo.
- Al termine, `desktop.screenshot` per verificare il risultato. Il consulente vede il confronto before/after nella dashboard.

Risultato: Da 10-15 minuti a 90 secondi. Eliminazione degli errori di battitura. Il consulente approva una volta il pattern,

poi va in automatico per i casi successivi identici.

Caso 3: Monitoraggio Errori Software

Scenario: Un'azienda IT ha sviluppatori che lavorano con terminali, log viewer, IDE. Gli errori critici devono essere escalati velocemente al responsabile. Spesso i developer junior non riconoscono la gravità di certi stack trace.

Come funziona con il Desktop Bridge:

- L'agente monitora in background le finestre dei processi configurati come "monitoraggio log": cmd.exe nella allowlist solo per lettura, terminali di IntelliJ, output console di Visual Studio.
- Esegue desktop.readWindow periodicamente (ogni 30 secondi) o quando rileva un cambiamento significativo nel contenuto.
- Il testo estratto viene analizzato dall'LLM con context dalla KB interna: "questo NullPointerException nella classe OrderService corrisponde a un bug noto documentato in KB#4421. La soluzione è nel branch hotfix/ITASV-2341."
- L'agente mostra il risultato nella barra laterale: "Errore riconosciuto. Vedere KB#4421 per la soluzione. Vuoi che apra il documento?"
- Se il developer risponde sì, desktop.focusWindow apre il browser sul documento KB corrispondente.
- Se l'errore è critico e non ha soluzione nota, l'agente crea automaticamente un ticket Jira (via API, non via Desktop Bridge) e notifica il consulente.

Risultato: Mean time to identify (MTTI) ridotto da ore a minuti. I developer junior hanno supporto contestuale immediato. Le issue critiche vengono escalate automaticamente.

Caso 4: Data Entry da Documenti

Scenario: Studio commercialista con 200+ clienti. Ogni mese ricevono fatture di acquisto dei clienti (PDF fisici scansionati o digitali) e devono inserirle manualmente nel gestionale contabile.

Come funziona con il Desktop Bridge:

- Il dipendente apre una fattura PDF nel viewer. L'agente rileva l'apertura di un PDF.
- Esegue desktop.analyzeScreen con domanda specifica: "Estrai da questa fattura: fornitore, P.IVA fornitore, numero fattura, data, importo imponibile, IVA, totale, descrizione beni/servizi".
- L'LLM con vision analizza lo screenshot del PDF e restituisce un JSON strutturato con tutti i dati.
- L'agente apre il gestionale contabile, naviga a "Inserimento fattura acquisto".
- Compila campo per campo con desktop.typeText e desktop.clickElement per le dropdown (es. aliquota IVA).
- Il dipendente o il consulente approva il set di azioni prima dell'inserimento, oppure il consulente ha pre-approvato il pattern per questo specifico gestionale.
- Audit trail completo con screenshot pre/post per ogni fattura inserita.

Risultato: Da 5-8 minuti per fattura a 30-45 secondi. Per uno studio con 500 fatture/mese, risparmio di ~35-40 ore/mese di lavoro manuale. Riduzione errori di trascrizione dal tipico 2-3% a <0.1%.

Caso 5: Meeting Assistant

Scenario: Team di vendita con 5 commerciali che fanno 3-4 call Teams/Zoom al giorno. Dopo ogni call, i commerciali devono compilare manualmente il CRM con i punti salienti della conversazione, gli impegni presi, il next step. Spesso questo non viene fatto puntualmente o è incompleto.

Come funziona con il Desktop Bridge:

- L'agente rileva l'apertura di Microsoft Teams o Zoom (processi nella allowlist).
- Monitora la finestra della trascrizione in tempo reale (Teams mostra la trascrizione in live se abilitata). Esegue `desktop.readWindow` ogni 60 secondi per aggiornare il contesto.
- Incrocia le informazioni con la KB: riconosce i nomi dei clienti menzionati, i prodotti, i prezzi, le date.
- Al termine della call (l'agente rileva la chiusura della finestra Teams o il cambio di stato), genera automaticamente:
 - Riepilogo della call (3-5 bullet)
 - Action items con assegnatario e data se menzionata
 - Bozza di nota CRM
 - Suggerimento next step
- Il commerciale rivede il riepilogo nella barra laterale e con un click "Inserisci in CRM" lancia il flusso di data entry automatico nel CRM (Caso 2).
- Opzionalmente, l'agente suggerisce di aggiornare il calendario con il follow-up concordato durante la call.

Risultato: Il CRM viene aggiornato entro 2 minuti dalla fine di ogni call invece di "domani se ci ricordo". La qualità delle note migliora perché non dipende dalla memoria del commerciale dopo 4 call consecutive.

Caso 6: Assistente Sviluppatore

Scenario: Team di sviluppo in una software house. I developer lavorano in VS Code o IntelliJ su un progetto con documentazione tecnica specifica (architettura interna, convenzioni, API custom). Ogni nuova feature richiede di consultare documenti distribuiti su Confluence, Jira, wiki interne.

Come funziona con il Desktop Bridge:

- L'agente monitora la finestra dell'IDE. Esegue `desktop.readWindow` periodicamente o quando l'utente chiede esplicitamente aiuto.
- Legge il file attivo, la funzione selezionata, i messaggi di errore nel terminale integrato.
- Incrocia con la KB del progetto: pattern architetturali documentati, convenzioni di naming, esempi di codice approvati, ADR (Architecture Decision Records), note su librerie interne.
- Suggerisce nella barra laterale: "Questa funzione assomiglia a `PaymentService.process()` — vedi ADR-041 per il pattern raccomandato. Vuoi vedere l'esempio?"
- Se il developer chiede "scrivi il boilerplate per un nuovo Repository secondo le nostre convenzioni", l'agente genera il codice e, con approvazione, lo inserisce direttamente nell'IDE con `desktop.typeText`.
- Quando l'agente rileva un errore nel terminale, lo analizza e cerca in KB se è un errore noto con soluzione documentata.

Risultato: I developer junior diventano produttivi prima (non devono memorizzare tutte le convenzioni). I developer senior risparmiano tempo sulle code review di base (l'agente suggerisce le correzioni prima del commit). La coerenza del codebase migliora.

6. Configurazione e Setup

Installazione Local Agent su Windows

Prerequisiti:

- Windows 10 (1903+) o Windows 11
- .NET Framework 4.8+ (per UI Automation)
- 4 GB RAM disponibili, 500 MB spazio disco
- Connessione internet per comunicazione con il gateway

Procedura:

- Download: Il consulente invia al cliente un link personalizzato contenente l'installer pre-configurato con il tenantId e il gatewayUrl. Il link è univoco e scade dopo 48 ore.
- Installazione: Eseguire 108VisionAI-Setup-{versione}.exe. L'installer:
 - Installa l'applicazione Electron in %LOCALAPPDATA%\108VisionAI
 - Crea la configurazione iniziale in %APPDATA%\108VisionAI\config.json
 - Registra l'autostart (a scelta dell'utente)
 - Richiede le autorizzazioni necessarie (vedi sotto)
- Permessi richiesti durante installazione:
 - "Consenti a 108 Vision AI di leggere il contenuto delle finestre?" → Necessario per Accessibility API → Richiesto
 - "Consenti a 108 Vision AI di simulare input tastiera e mouse?" → Necessario per azioni high-risk → Opzionale (solo se si vogliono le azioni, non solo la lettura)
 - "Avvia automaticamente all'avvio di Windows?" → Consigliato
- Prima configurazione: Al primo avvio, l'agente mostra una finestra di benvenuto che chiede:
 - Nominativo dell'utente (per i log di audit)
 - Conferma del gateway URL (pre-compilato dall'installer)
 - Test di connessione al gateway (pulsante "Verifica connessione")
- Attivazione Desktop Access: Dopo la connessione, cliccare sull'icona nella system tray → "Attiva Desktop Bridge" → Inserire il codice di attivazione fornito dal consulente.

Abilitazione Desktop Access

L'icona nella system tray di Windows mostra lo stato corrente del Desktop Bridge:

Colore | Stato | Significato

Verde pieno | CONNESSO + ATTIVO | Agente connesso al gateway, Desktop Bridge abilitato, pronto a ricevere comandi

Verde vuoto (outline) | CONNESSO, desktop disabilitato | Agente connesso ma Desktop Bridge disabilitato (solo funzionalità chat/KB)

Giallo | CONNESSIONE DEGRADATA | Agente connesso ma latenza alta o WebSocket instabile

Arancione | APPROVAZIONE IN ATTESA | C'è un'azione high-risk in attesa di approvazione dal consulente

Rosso | DISCONNESSO | Agente non raggiunge il gateway. Controllare internet/VPN

Grigio | AGENTE SOSPESO | Rate limit superato o sospensione manuale

Menu tray click destro:

- Stato connessione e versione
- Attiva/Disattiva Desktop Bridge (toggle rapido)
- Sospendi per 15/30/60 minuti (utile per attività sensitive che non si vogliono monitorare)
- Apri barra laterale agente

- Storico azioni locali
- Impostazioni
- Esci

Configurazione processi consentiti

Il file %APPDATA%\108VisionAI\config.json può essere editato manualmente o tramite il pannello Impostazioni dell'agente:

```
{
  "tenantId": "tenant_xyz_cliente",
  "gatewayUrl": "wss://gateway.108vision.ai/ws",
  "agentUserId": "dipendente.cognome@azienda.it",
  "desktopAccess": {
    "enabled": true,
    "strictMode": true,
    "processAllowList": [
      "outlook.exe",
      "chrome.exe",
      "msedge.exe",
      "firefox.exe",
      "winword.exe",
      "excel.exe",
      "crm_pro.exe",
      "Code.exe",
      "devenv.exe"
    ],
    "processBlockList": [
      "keepass.exe",
      "1password.exe",
      "bitwarden.exe",
      "LastPass.exe",
      "taskmgr.exe",
      "regedit.exe",
      "powershell.exe",
      "cmd.exe",
      "wt.exe"
    ],
    "rateLimits": {
      "actionsPerMinute": 30,
      "highRiskPerMinute": 10,
      "screenshotsPerMinute": 20
    },
    "approvalTimeout": 60,
    "requireLocalConfirmForHighRisk": false
  },
  "sidebarEnabled": true,
  "autoStartEnabled": true,
  "logLevel": "info"
}
```

Nota: Le modifiche a processBlockList che tentano di rimuovere elementi dalla system blocklist vengono ignorate silenziosamente. La system blocklist (password manager, tool di sistema) è hardcoded nell'agente e non può essere rimossa dalla configurazione.

Configurazione dal lato Dashboard

Il consulente configura il livello di autonomia per ogni cliente dalla dashboard web:

Sezione "Desktop Bridge Settings" per ogni cliente:

- Livello di Autonomia Globale:
 - SUPERVISED — tutte le azioni high-risk richiedono approvazione esplicita
 - STANDARD — auto-approve per pattern pre-registrati, high-risk per nuovi pattern
 - TRUSTED — solo le azioni esplicitamente etichettate come "always-ask" richiedono approvazione
- Pattern Pre-approvati: Il consulente può registrare template di azioni che vengono eseguite senza approvazione ogni volta:

Nome: "Data entry email Outlook"
 Condizione: typeText in finestra outlook.exe in campo "reply body"
 Pre-approvato: SI
 Limite: max 5 volte/giorno

- Notifiche: Canali di notifica per le approvazioni richieste (email, mobile push, webhook Slack/Teams).
- Orari operativi: Fasce orarie in cui il Desktop Bridge è attivo (es. solo in orario lavorativo 8:00-19:00).
- Screenshot Policy: Retention locale (giorni), retention cloud (giorni), privacy mode (oscuramento automatico di certi pattern di testo).

7. Dashboard del Consulente

Monitor Desktop

La sezione "Monitor Desktop" nella dashboard permette al consulente di avere visibilità in tempo (quasi) reale su cosa sta facendo il cliente con il Desktop Bridge.

Componenti principali:

Finestra attiva: Un riquadro che mostra il titolo e il processo della finestra attualmente in primo piano sul computer del cliente. Aggiornamento ogni 30 secondi (non in tempo reale per minimizzare bandwidth e rispettare la privacy).

Ultimo screenshot: Thumbnail dell'ultimo screenshot acquisito dall'agente. Click per ingrandire. Timestamp dell'acquisizione. Il consulente può richiedere un screenshot "fresh" con il bottone "Aggiorna" — questo esegue desktop.screenshot in tempo reale.

Stato connessione: Badge con latenza WebSocket, uptime della sessione corrente, versione agent installata.

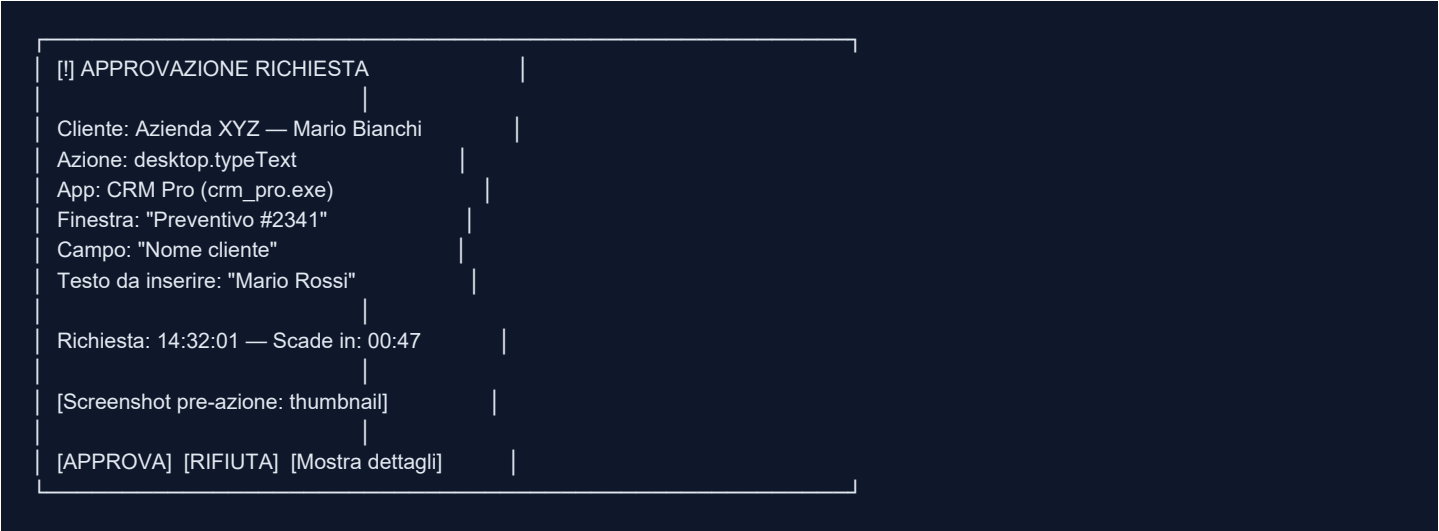
Azioni recenti: Lista delle ultime 10 azioni eseguite (o tentate), con esito e tipo. Click su ognuna per vedere i dettagli e gli screenshot.

Indicatori di attività: Grafico a barre delle azioni per ora nelle ultime 24h. Utile per capire i pattern di utilizzo e identificare picchi anomali.

Coda Approvazioni

La coda approvazioni è il componente più time-critical della dashboard. Deve essere accessibile rapidamente, anche da mobile.

Struttura di ogni item in coda:



Countdown: Il timer di 60 secondi è visibile e urgente. Sotto i 10 secondi, il bordo diventa rosso pulsante.

Approvazione bulk: Se ci sono più azioni della stessa tipologia in coda (es. compilazione di 10 campi dello stesso form, pre-pianificate dall'agente), il consulente può approvare tutte con un click, dopo aver verificato il piano d'azione complessivo mostrato in una modale di riepilogo.

Rifiuto con motivazione: Il consulente può selezionare una motivazione rapida dal rifiuto (lista dropdown: "Finestra sbagliata", "Dati errati", "Azione non necessaria", "Sicurezza", "Altro") o scrivere un testo libero che viene loggato e inviato all'agente come feedback.

Screenshot Viewer

Il visualizzatore di screenshot supporta:

Zoom e pan: Ingrandimento fino a 400% con trascinamento per navigare in immagini grandi (es. schermi 4K).

Confronto before/after: Per ogni azione high-risk, un slider verticale permette di confrontare pre e post azione sullo stesso punto dello schermo. Utile per verificare che l'azione abbia prodotto l'effetto atteso.

Annotazioni: Il consulente può aggiungere annotazioni rettangolari o frecce sullo screenshot (utile per il feedback asincrono: "vedi campo evidenziato in rosso — quello corretto è più in basso").

Fullscreen: Modalità fullscreen per analisi dettagliata.

Download: Download dello screenshot in alta qualità per documentazione o ticket di supporto.

Privacy blur: Possibilità di oscurare manualmente aree dello screenshot prima di condividerlo (es. prima di allegarlo a un ticket Jira visibile a più persone).

Storico Azioni

Il log completo di tutte le azioni del Desktop Bridge per il cliente selezionato.

Filtri disponibili:

- Tipo azione (typeText, clickElement, ecc.)
- Livello rischio (read-only, low, high)
- Processo/App (outlook.exe, chrome.exe, ecc.)
- Esito (SUCCESS, FAILED, BLOCKED, REJECTED, TIMEOUT)
- Data/ora (range picker)
- Approvato da (filtro per consulente se ci sono più consulenti)

Per ogni entry nello storico:

- Timestamp
- Tipo azione e parametri (oscurati se contengono PII configurati)
- App target
- Livello rischio
- Esito
- Latenza esecuzione
- Links a screenshot pre/post
- Approvatore (per azioni high-risk)

Esportazione: CSV o JSON per periodi selezionati, utile per rendicontazione al cliente o analisi dei pattern di utilizzo.

Retention: 90 giorni online nella dashboard, fino a 1 anno in archivio (a seconda del piano).

8. Client macOS — Roadmap Fase 8

Sfide specifiche macOS

Il porting del Desktop Bridge su macOS non è banale. Apple ha introdotto negli ultimi anni un modello di permessi molto più restrittivo rispetto a Windows. Le principali sfide:

1. Accessibility Permission (TCC — Transparency, Consent, and Control)

Per leggere l'UI tree di un'altra applicazione, l'app deve richiedere il permesso "Accessibility" in System Settings > Privacy & Security > Accessibility. Il permesso viene richiesto al primo utilizzo, l'utente deve concederlo manualmente, e Apple può revocarlo con update del sistema.

Complicazione aggiuntiva: nel macOS con Hardened Runtime (richiesto per notarizzazione), certi metodi AX (Accessibility API) richiedono che l'app abbia il permesso esplicito. Le app Electron non sandboxate possono ottenerlo, ma il processo di notarizzazione di Apple può rifiutare app che usano certi entitlement.

2. Screen Recording Permission

Per catturare screenshot di altre applicazioni (diverso dalla propria finestra), macOS richiede il permesso "Screen Recording" in System Settings > Privacy & Security > Screen Recording. Senza questo permesso, `captureScreen()` restituisce uno screenshot nero o con le finestre di altre app oscurate.

3. Input Simulation

Su macOS, simulare input di tastiera e mouse verso altre applicazioni richiede il permesso Accessibility (lo stesso di sopra). Le API usate (`CGEventPost`) funzionano solo se il processo ha il permesso.

4. Notarizzazione Apple

Qualsiasi app distribuita fuori dall'App Store (e 108 Vision AI non può essere nell'App Store per via dei permessi elevati richiesti) deve essere notarizzata da Apple. Apple esegue scan automatici e rifiuta app che usano API "dangerous". La

combinazione di Screen Recording + Accessibility + Input Simulation è borderline e richiede un processo di notarizzazione attento.

5. System Integrity Protection (SIP)

SIP limita cosa si può fare con certi processi di sistema. Non dovrebbe impattare le funzionalità core del Desktop Bridge, ma limita il debugging durante lo sviluppo.

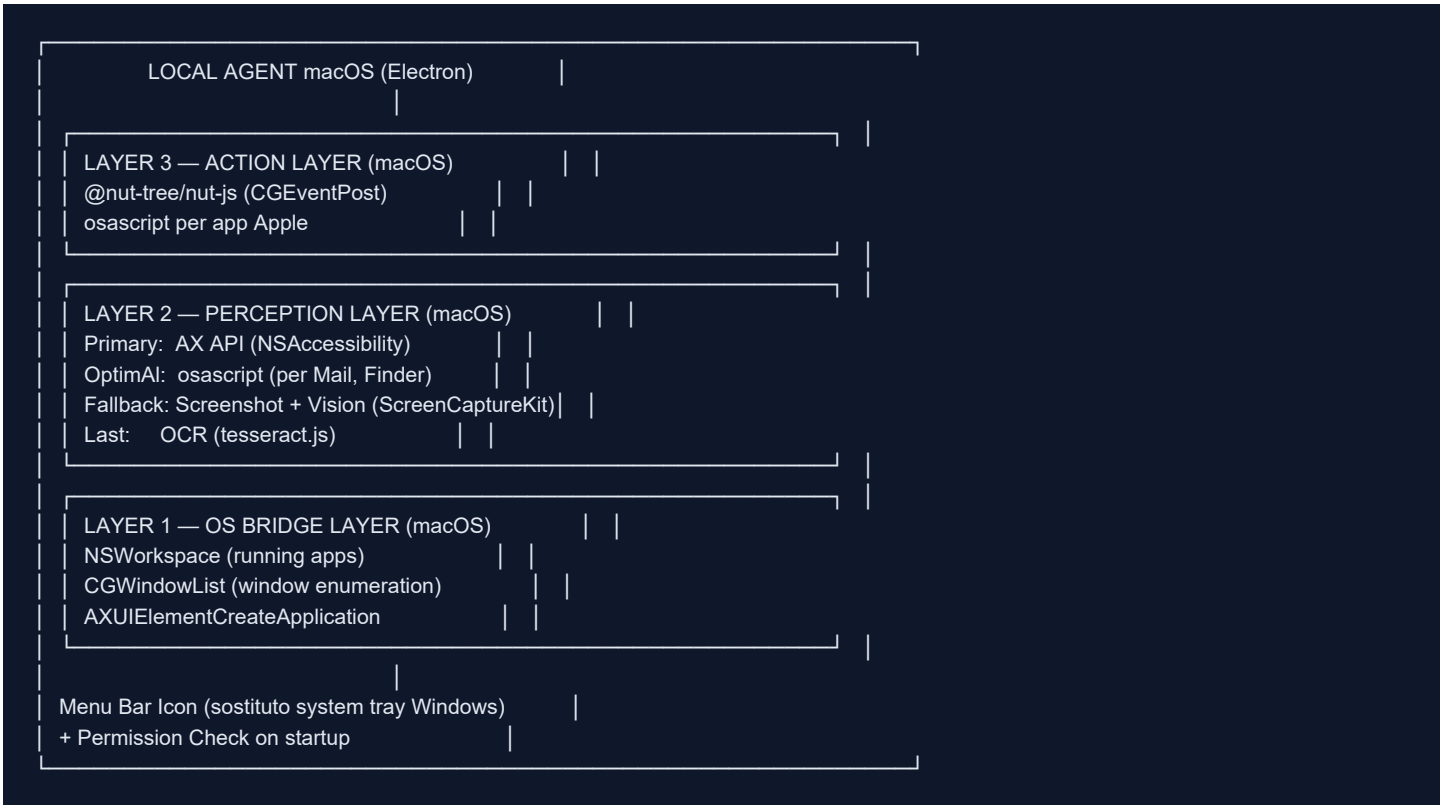
6. AppleScript vs AX API

Su macOS esistono due approcci per l'automazione:

- AppleScript / osascript: Potente per le app che lo supportano (principalmente app Apple: Mail, Finder, Safari, Numbers, Pages). Linguaggio specifico, non portabile.
- AX API (Accessibility): Equivalente macOS di UI Automation su Windows. Funziona con tutte le app, ma la qualità degli alberi di accessibilità varia.

La strategia sarà usare AX API come primario (consistente con l'approccio Windows), e osascript come ottimizzazione per le app Apple che lo supportano bene.

Architettura macOS



Provider macos.ts — il modulo specifico per macOS che implementa la stessa interfaccia del provider Windows (windows.ts), garantendo che il Security Manager e il resto del Local Agent funzionino identicamente su entrambe le piattaforme:

```
// Interfaccia comune (platform-agnostic)
interface DesktopProvider {
  listWindows(): Promise<WindowInfo[]>;
  readWindow(target: WindowTarget): Promise<WindowContent>;
  focusWindow(target: WindowTarget): Promise<boolean>;
  typeText(target: WindowTarget, field: string, text: string): Promise<ActionResult>;
  clickElement(target: WindowTarget, element: string): Promise<ActionResult>;
  screenshot(target: WindowTarget): Promise<ScreenshotResult>;
  checkPermissions(): Promise<PermissionStatus>;
}

// Implementazione macOS
class MacOSDesktopProvider implements DesktopProvider {
  async listWindows(): Promise<WindowInfo[]> {
    // Usa CGWindowListCopyWindowInfo via node-native-module
    const windows = await this.nativeModule.cgWindowList();
    return windows.map(this.mapCGWindowToWindowInfo);
  }

  async readWindow(target: WindowTarget): Promise<WindowContent> {
    // Tenta AX API prima
    const axResult = await this.readWithAxAPI(target);
    if (axResult.quality > AX_QUALITY_THRESHOLD) return axResult;

    // Fallback: osascript per app Apple
    if (this.isAppleApp(target.processName)) {
      return await this.readWithOsascript(target);
    }

    // Ultimo fallback: screenshot + vision
    return await this.readWithVision(target);
  }
}
```

Menu Bar Icon: Su macOS non esiste la "system tray" come su Windows. Si usa il Menu Bar (barra dei menu in alto). L'icona e il menu a discesa hanno la stessa funzionalità del tray Windows. Un piccolo indicatore colorato (dot) accanto all'icona mostra lo stato (verde/giallo/rosso).

Piano di implementazione

Durata totale stimata: 10-14 giorni lavorativi

Settimana 1 (Giorni 1-5): Core macOS Layer

Giorno	Task	Output
1	Setup ambiente macOS (macOS 14 Sonoma + Xcode + Node native), studio permessi TCC	Dev environment pronto
1-2	Implementare macos.ts Layer 1: CGWindowList, NSWorkspace, lista processi running	listWindows() funzionante
2-3	Implementare AX API wrapper in Swift/Obj-C bridge per Node.js (node-addon-api): AXUIElementCreateApplication, AXUIElementCopyAttributeValue	
3-4	Implementare readWindow() con AX API + quality scoring	readWindow() su almeno Chrome, Safari, Mail
4-5	Implementare focusWindow() via NSRunningApplication.activate()	focusWindow() funzionante
5	Test end-to-end Layer 1+2 con agente connesso al gateway di test	Comunicazione WebSocket + read funzionanti

Settimana 2 (Giorni 6-10): Action Layer + Permission Flow + Build

Giorno | Task | Output

6-7 | Implementare Action Layer: @nut-tree/nut-js su macOS per typeText, clickElement, pressHotkey | Azioni funzionanti su app di test

7 | Implementare osascript bridge per ottimizzare interazione con Mail, Finder | typeText ottimizzato su Mail.app

8 | Implementare Permission Check Flow: check all'avvio per Accessibility + Screen Recording, guida utente step-by-step per concedere i permessi |

8-9 | Menu Bar icon con indicatori di stato, toggle Desktop Access, menu contestuale | System tray equivalente macOS

9-10 | Window Guard su macOS (diverso da Windows: usa windowNumber di CGWindow invece di HWND), Rate Limiter, Security Manager test | Modello sicu

10 | Build Electron macOS, firma codice (Developer ID), test su macOS 13 Ventura + 14 Sonoma | Build firmata funzionante

Giorni 11-14 (Buffer): Notarizzazione + QA + Documentazione

Giorno | Task | Output

11 | Submission a Apple Notary Service, attesa (tipicamente 15-30 min ma possibili problemi con entitlement) | App notarizzata o feedback da Apple

11-12 | Fix eventuali problemi notarizzazione, re-submission | App notarizzata

12-13 | QA su macOS: test tutte le 12 azioni, test permessi revocati/concessi, test rate limit, test Window Guard, test approvazione remota | QA r

13-14 | Documentazione tecnica specifica macOS, aggiornamento MANUALE-Desktop-Bridge.md, release notes | Documentazione aggiornata

Differenze rispetto a Windows

Aspetto | Windows | macOS

Identificatore finestra | HWND (intero 32/64-bit) | windowNumber (CGWindowID, intero)

Accessibilità API | UI Automation (COM) | NSAccessibility (AX API)

Enumerazione finestre | EnumWindows (Win32) | CGWindowListCopyWindowInfo

Screenshot | node-screenshots con HWND | ScreenCaptureKit (macOS 12.3+) o CGWindowListCreateImage

Input simulation | SendInput / @nut-tree | CGEventPost / @nut-tree

System tray | NotifyIcon (Windows) | NSStatusItem (Menu Bar)

Permessi | Richiesti a install time, generalmente auto-concessi | TCC: devono essere concessi esplicitamente dall'utente in System Settings

Automazione app native | PowerShell / COM | AppleScript / osascript

Processi di sistema bloccati | Task Manager, Registry Editor | — (più aperto)

Distribuzione | MSIX o EXE installer | DMG con app firmata + notarizzata

Firma codice | Code Signing Certificate (Sectigo, DigiCert) | Apple Developer ID (costa \$99/anno)

Dal lato utente, le differenze percepite sono minime:

- L'icona nella menu bar (in alto) invece che nella tray (in basso a destra)
- Il dialog di primo avvio chiede di andare in System Settings per concedere i permessi (2 permessi invece di 1 su Windows)
- Le shortcut da tastiera nei comandi automatici usano cmd invece di ctrl dove appropriato

9. Vantaggi per il Cliente

ROI misurabile

I vantaggi del Desktop Bridge non sono teorici. Ogni caso d'uso ha metriche precise:

Risparmio di tempo per categoria di task:

Task	Tempo senza Desktop Bridge	Tempo con Desktop Bridge	Risparmio
Risposta email standard con KB	5-7 min	45-90 sec	~80%
Data entry da PDF a gestionale	8-12 min	60-90 sec	~85%
Compilazione CRM dopo call	10-15 min	2-3 min	~80%
Data entry fattura acquisto	5-8 min	30-45 sec	~87%
Ricerca informazione in KB per rispondere cliente	8-15 min	30-60 sec	~90%
Compilazione form ripetitivo	3-5 min	20-30 sec	~88%

Calcolo ROI esempio per una PMI tipo:

Scenario: azienda B2B, 5 dipendenti che gestiscono clienti
Tasks quotidiani automatizzabili:
- 20 email risposta/giorno × 5 min risparmio = 100 min/giorno
- 5 CRM update/giorno × 8 min risparmio = 40 min/giorno
- 10 data entry vari/giorno × 3 min risparmio = 30 min/giorno

Totale: 170 minuti/giorno = 2h 50min

Ore risparmiate/mese (22 giorni lavorativi): ~62 ore
Costo orario dipendente tipico: €25/ora
Risparmio mensile stimato: €1.550

Costo piano 108 Vision AI (5 utenti): €250-400/mese
ROI mensile: €1.150-1.300
Payback: < 30 giorni

Confronto Before/After

Caso pratico: Ufficio commerciale di una PMI manifatturiera

Attività	Prima del Desktop Bridge	Dopo il Desktop Bridge	Risparmio
Risposta email richiesta preventivo	12 min (leggi, cerca in catalogo, calcola, scrivi, correggi)	2 min (leggi, approva bozza AI)	83%
Invio preventivo dopo approvazione	5 min (copia dati da email a gestionale, genera PDF)	45 sec (agente compila il form, genera PDF)	85%
Follow-up cliente dopo 7 giorni	Spesso dimenticato	Automatico con reminder	Da 40% efficacia a 95%
Aggiornamento CRM dopo call	8 min (spesso fatto male o non fatto)	1.5 min (agente genera note, dipendente approva)	81%
Trovare precedente comunicazione con cliente	6-10 min (ricerca in archivio email)	30 sec (agente cerca in KB integrata)	92%
Inserimento ordine da email cliente	10-15 min (data entry manuale nel gestionale)	2 min (agente legge email, compila form)	85%
Risposta a domanda tecnica su prodotto	15-20 min (cerca in catalogo tecnico, documenti)	1-2 min (agente cerca in KB, suggerisce risposta)	

Sicurezza e fiducia

Il modello ad approvazione remota del Desktop Bridge è progettato per generare fiducia, non solo automazione. I clienti SMB che considerano strumenti di AI automation hanno spesso preoccupazioni legittime:

"E se l'AI fa qualcosa di sbagliato?"

Con il Desktop Bridge, le azioni ad alto rischio non vengono mai eseguite senza che un essere umano (il consulente) le abbia approvate, vedendo uno screenshot di esattamente cosa sta per succedere. Questo è fondamentalmente più sicuro di qualsiasi script di automazione tradizionale che gira in cieco.

"E se i dati del cliente finiscono nell'AI?"

La KB aziendale è privata per tenant. I dati non vengono condivisi tra clienti diversi. I screenshot di audit vengono conservati cifrati e accessibili solo al consulente e all'admin. Il modello LLM non "impara" dai dati del cliente (inference-only, nessun fine-tuning su dati aziendali senza consenso esplicito).

"E se l'AI prende il controllo del computer?"

Il Local Agent ha un kill switch immediato: il dipendente può disabilitare il Desktop Bridge in un click dall'icona nella tray. L'agente non può agire senza connessione al gateway. Se la connessione si interrompe, tutte le azioni vengono sospese. Il rate limiting previene qualsiasi scenario di azioni massicce non controllate.

"Chi è responsabile se qualcosa va storto?"

Il consulente che approva un'azione è il responsabile di quella approvazione. L'audit trail completo con timestamp e nome dell'approvatore garantisce tracciabilità piena. Questo è un vantaggio anche per il cliente: in caso di errore, si può capire esattamente cosa è successo e chi ha autorizzato cosa.

10. Limitazioni e Roadmap

Limitazioni attuali

1. App senza accessibility tree

Alcune applicazioni legacy o custom non espongono un albero di accessibilità decente. In questi casi il fallback vision (screenshot + LLM) funziona per la lettura, ma le azioni di click/type sono meno precise perché basate su coordinate visive stimate dall'LLM, non su handle di elemento precisi. Impatto: azioni possibili ma con rischio più alto di click sbagliato.

2. Latenza vision

Quando si usa il fallback vision (screenshot + analisi LLM), la latenza è nell'ordine di 1-3 secondi per ogni percezione, contro i 50-200ms dell'Accessibility API. Per workflow ad alta frequenza (es. 10 campi form da compilare in sequenza), la differenza è percepibile. Impatto: per app problematiche, il flusso è più lento.

3. Azioni multi-step con dipendenze complesse

Il sistema attuale esegue azioni singole (o sequenze pre-definite). Non supporta ancora workflow dove il passo successivo dipende dall'output del passo precedente in modo ramificato (es. "se il campo 'Tipo cliente' ha valore A vai a pagina 2, se ha valore B vai a pagina 3"). Questi workflow richiedono logica di planning che è in roadmap ma non ancora implementata.

4. Solo Windows (al 9 giugno 2026)

La Fase 8 macOS è pianificata. Linux non è ancora in roadmap per il client desktop.

5. Finestre browser con contenuto web

Per le web app aperte in Chrome/Edge/Firefox, la Accessibility API fornisce un tree generico del browser ma non sempre il contenuto specifico della pagina (dipende dall'implementazione ARIA della web app). Spesso il fallback vision è necessario per leggere correttamente una web app. Impatto: per CRM e gestionali web, la qualità di percezione può variare.

6. App a schermo intero o con protezioni anti-screenshot

Alcune app bloccano attivamente i screenshot (es. app bancarie, DRM media player). Il Desktop Bridge non può aggirare queste protezioni — né sarebbe corretto farlo. Impatto: queste app non sono utilizzabili con Desktop Bridge e vengono automaticamente messe nella blocklist.

7. Coordinamento multi-monitor

Su setup multi-monitor, alcune funzionalità come focusWindow e le coordinate degli screenshot possono avere comportamenti non ottimali se le finestre sono su monitor con DPI diversi. Il supporto multi-monitor è parziale e in miglioramento.

Roadmap futura

Fase 8 — macOS Client (Q3 2026, 10-14 giorni)

Portabilità completa su macOS come descritto nella sezione 8. Parità funzionale con Windows per tutte le 12 azioni. Distribuzione via DMG firmato e notarizzato.

Fase 9 — Linux Client (Q4 2026)

Supporto Ubuntu 22.04+ e Debian 12+. Usa AT-SPI2 (Assistive Technology Service Provider Interface) come equivalente Linux di UI Automation. Input simulation via xdotool (X11) e ydotool (Wayland). Distribuzione via .deb e Applmage. Rilevanza per clienti con workstation Linux (prevalentemente settore IT, sviluppo, sistemisti).

Fase 10 — Workflow Multi-Step con Planning (Q1 2027)

Implementazione di un motore di planning leggero che permette di definire workflow condizionali:

```
IF campo "Tipo cliente" = "Enterprise"
  THEN naviga a tab "Contratti"
  AND compila campo "SLA" con "24h"
ELSE
  compila campo "SLA" con "48h"
```

Questi workflow possono essere definiti graficamente nella dashboard e salvati come "macro approvate" per i clienti. Riduce drasticamente il numero di approvazioni necessarie per operazioni complesse ricorrenti.

Fase 11 — Integrazione Diretta API App (Q2 2027)

Per le app che offrono API native (Microsoft Teams, Slack, Microsoft 365, Google Workspace, Salesforce, HubSpot), utilizzare le API dirette invece del Desktop Bridge quando possibile. Vantaggi: più veloce, più affidabile, non dipende dalla UI, funziona anche con app non visibili.

Il Desktop Bridge rimane il fallback universale per app senza API, e il meccanismo di approvazione rimane invariato.

Fase 12 — Offline Mode e Edge AI (Q3 2027)

Modalità di funzionamento parziale senza connessione al cloud: il Local Agent usa un modello LLM locale (es. Llama 3 quantizzato) per le operazioni di bassa complessità (lettura finestra, estrazione testo strutturato, risposte a domande semplici sulla KB locale). Le operazioni ad alta complessità e le approvazioni rimangono cloud-dependent.

Vantaggi: funzionamento in ambienti con connettività limitata, riduzione latenza per operazioni semplici, maggiore resilienza.

Fase 13 — Estensione Browser (Q4 2027)

Un'estensione Chrome/Edge/Firefox che fornisce all'agente accesso diretto al DOM delle pagine web, senza dover passare dall'Accessibility API del browser. L'estensione:

- Espone un endpoint locale (comunicazione via `chrome.runtime.sendMessage`)
- Permette lettura diretta del DOM (struttura, valori, stati)
- Permette interazione precisa con elementi web tramite selettori CSS/XPath
- Funziona anche su pagine che bloccano l'accessibilità a livello OS

Questo risolve il problema della qualità variabile del perception layer per web app e rende le azioni su CRM web, gestionali cloud e form web molto più affidabili.

Manuale Desktop Bridge — 108 Vision AI — Versione 1.0 — 9 giugno 2026

Per aggiornamenti: contattare il team prodotto 108 Vision AI